Bound-Propagation Robustness Verification of an MNIST Classifier: α, β-CROWN vs. Marabou

Jaemin Paek

Abstract

We verify the local ℓ_∞ robustness of a small fully-connected MNIST classifier with α, β-CROWN and compare it, on identical instances, with the SMT-based verifier Marabou used in a previous study. The two tools agree on every shared verdict and on per-sample robustness thresholds, which we use as a soundness cross-check. The interesting differences are quantitative: the speed gap is not constant but opens up at a specific ε, the verification effort shifts from cheap bounds to branch-and-bound within a narrow ε band, all counterexamples are found by the attack rather than by search, and the choice of branching heuristic — not the solver itself — decides success on hard instances.

1 Setup

Model and data. The network is a fully-connected classifier $784 \to 64 \to 32 \to 10$ with ReLU activations (96 hidden units), exported to ONNX and reused unchanged from the Marabou study so that both tools verify *the same* network on *the same* inputs. Inputs are MNIST test images, normalized with mean 0.1307 and std 0.3081 (pixel values lie in $[-0.4242, 2.8215]$).

Property. For an input x with true label L and radius ε, we verify local ℓ_∞ robustness: for all x' with $\|x' - x\|_\infty \le \varepsilon$ (clipped to the valid range), the network keeps L as the arg-max. We encode this per instance as VNNLIB, with the perturbation applied in the *normalized* input space to match the Marabou queries exactly; UNSAT $\Rightarrow$ verified, SAT $\Rightarrow$ falsified. α, β-CROWN runs on CPU with branch-and-bound (no Gurobi) and the `kfsb` branching heuristic.

2 Results and Interpretation

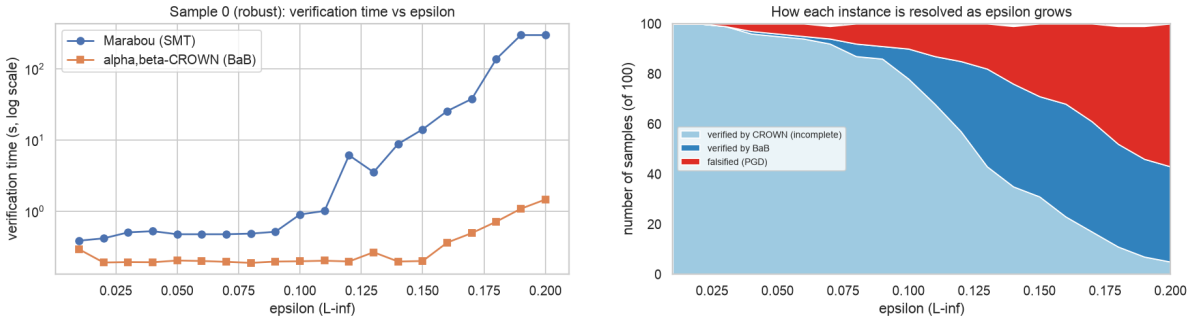
Phase 1 — scan (100 samples, $\varepsilon = 0.05$). 96 verified robust, 4 falsified (samples 8, 33, 92, 96), 0 timeouts; mean 0.21 s/instance.

Phase 2 — thresholds. The smallest falsified ε is 0.03 for sample 8 and 0.05 for sample 33; sample 0 stays robust through $\varepsilon = 0.20$. These match the Marabou thresholds exactly.

The speed advantage is ε-dependent, not constant. On the robust sample 0, the two tools are essentially tied at small radii ($\varepsilon = 0.01$: Marabou 0.39 s, α, β-CROWN 0.30 s) and remain comparable up to $\varepsilon = 0.11$ (Marabou 1.0 s). Marabou then degrades sharply — 6.2 s at $\varepsilon = 0.12$ and its 300 s timeout at $\varepsilon \ge 0.19$ — while α, β-CROWN rises only to 1.5 s at $\varepsilon = 0.20$ (Fig. 1a). The often-quoted ~270× gap is therefore a property of the *tail*; the honest summary is "matched until $\varepsilon \approx 0.11$, after which SMT search falls off a cliff."

Where the effort goes, and a sharp transition. Decomposing how each of the 100 instances is settled (Fig. 1b) shows that cheap incomplete CROWN bounds suffice for all 100 at $\varepsilon \le 0.02$, but only 5 at $\varepsilon = 0.20$; the branch-and-bound share rises steeply in a narrow band around $\varepsilon \approx 0.12$–0.15. The verification difficulty is concentrated, not spread evenly across ε.

All counterexamples come from the attack, never from search. Across every ε, falsified instances are resolved by the PGD attack (`unsafe-pgd`) and never by branch-and-bound (`unsafe-bab` = 0). For this network the cheap attack is already strong enough to falsify; the expensive complete search is spent almost entirely on *proving* robustness. Recovered counterexamples are clear misclassifications (e.g., sample 8: $5 \to 6$; 33: $4 \to 0$; 92: $9 \to 4$; 96: $1 \to 9$), each a small perturbation inside the ε-ball (Fig. 2).



(a) **(b)**

Figure 1: (a) Verification time vs. ε for robust sample 0 (log scale): comparable until $\varepsilon \approx 0.11$, then Marabou times out while α, β-CROWN stays near 1.5 s. (b) How the 100 instances are resolved as ε grows; the branch-and-bound share rises sharply around $\varepsilon \approx 0.12$–0.15.

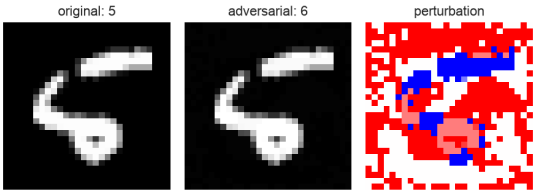


Figure 2: Counterexample for sample 8 at $\varepsilon = 0.05$ (original | adversarial | perturbation): an ℓ_∞ perturbation flips the prediction $5 \to 6$.

3 Comparison with Marabou

	α, β-CROWN	Marabou
Method	linear relaxation + branch-and-bound	SMT (Reluplex)
Interface	declarative YAML + VNNLIB	imperative Python API
Verdicts on shared instances	identical	identical
Robust sample, $\varepsilon = 0.20$	~1.5 s	300 s (timeout)
Single small query (cold start)	~2.5 s	~0.5 s

Beyond the table, three points are worth making from the experiments. First, the *agreement* itself is a result: matching the property across tools required applying ε in the normalized space inside the VNNLIB, and the identical thresholds (0.03, 0.05) confirm both that the encoding is correct and that the two independent engines are mutually consistent. Second, "ease of use" cut both ways: α, β-CROWN's config-driven batch mode made a 100-sample sweep a single run, but reaching a working setup required handling tool-specific friction (a mandatory `gurobipy` import even when Gurobi is unused, and a CPU-only build), whereas Marabou's Python API is more direct for a one-off query. Third, the recovered *counterexamples need not match*: Marabou's non-strict violation encoding (output$[j] \geq$ output$[L]$) can return a point whose arg-max ties the true class, while α, β-CROWN's PGD returns a strict misclassification — both are valid witnesses of the same non-robustness.

4 Strengths and Limitations

Strengths. α, β-CROWN scales gracefully where SMT does not: it certified all 100 test samples cheaply (enabling a full robustness-radius distribution — 43/100 remain robust beyond $\varepsilon = 0.20$), and it resolved the large-ε robust cases that drove Marabou to timeout.

Limitations observed. (i) It is *not* universally faster: a single small query costs ~2.5 s cold versus Marabou's ~0.5 s, because per-process start-up dominates for tiny models; the advantage appears only once start-up is amortized over a batch. (ii) Success can hinge on the branching heuristic rather than the solver: on the hardest instance (sample 22, $\varepsilon = 0.2$), `kfsb` verified in 31 s, `fsb` timed out, and the `babsr` heuristic crashed with an upstream `AttributeError`. The default `kfsb` worked, but the configuration surface is large and not uniformly robust.